OBJECT ORIENTED PROGRAMMING

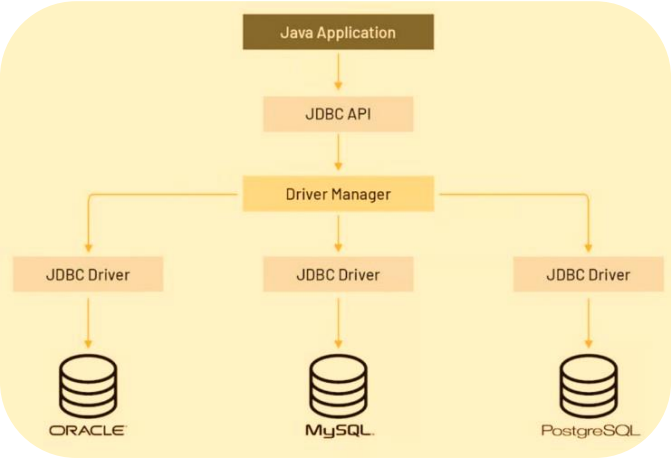
UNIT - V

AI303PC: OBJECT ORIENTED PROGRAMMING

USING JAVA

JAVA DATABASE CONNECTIVITY (JDBC)

- **Definition**
  - JDBC (Java Database Connectivity) is a standard Java API that provides a set of classes and interfaces to connect Java applications to relational databases (MySQL, Oracle, PostgreSQL) and perform database operations.
- **What JDBC Does**



JDBC Capabilities

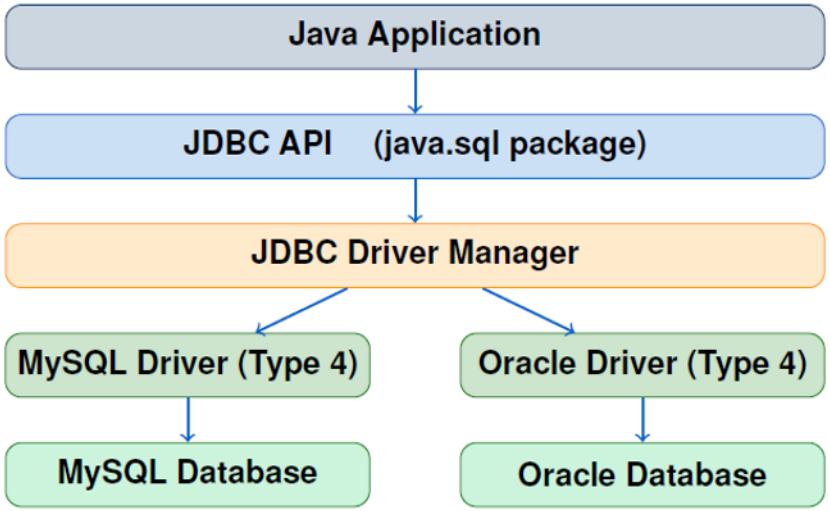
Operation	SQL Command
Create Table	CREATE TABLE
Insert Data	INSERT INTO
Read Data	SELECT
Update Data	UPDATE
Delete Data	DELETE

TYPES OF JDBC DRIVERS

- **Definition**
  - A JDBC Driver is a software component that translates JDBC calls into database- specific calls that the database can understand.

Type	Name	How It Works	Advantage	Disadvantage
1	JDBC-ODBC Bridge	Converts JDBC to ODBC calls	Works with any ODBC DB	Slow, Windows only
2	Native-API Driver	Converts JDBC to native DB calls	Faster than Type 1	Needs native library
3	Network Protocol	Sends to middleware server	No client installation	Needs middleware
4	Thin Driver ✓	Direct DB protocol conversion	Pure Java, fastest	DB-specific JAR needed

JDBC ARCHITECTURE



Key Point

Two-Tier: Java App → JDBC Driver → Database (Direct connection)
Three-Tier: Java App → Middleware → JDBC Driver → Database (via Application Server)

JDBC CLASSES AND INTERFACES

Important Classes & Interfaces in `java.sql`: Key Methods

Name	Type	Description
`DriverManager`	Class	Manages drivers; provides `getConnection()`
`Connection`	Interface	Represents a connection to the database
`Statement`	Interface	Executes simple SQL queries
`PreparedStatement`	Interface	Executes pre-compiled parameterized queries
`CallableStatement`	Interface	Executes stored procedures
`ResultSet`	Interface	Holds data returned by SELECT queries
`ResultSetMetaData`	Interface	Info about ResultSet columns
`DatabaseMetaData`	Interface	Info about the database itself
`SQLException`	Class	Handles database-related exceptions

Quick Reference:

```
1 DriverManager . getConnection(url , user , pass ); // Get connection
2 con . createStatement(); // Create statement
3 stmt . executeQuery(" SELECT ... "); // Returns ResultSet
4 stmt . executeUpdate(" INSERT / UPDATE / DELETE ... "); // Returns int
5 rs. next(); rs. getString(" col"); rs.getInt(" col ");
```

BASIC STEPS IN DEVELOPING A JDBC APPLICATION

1. Load the JDBC Driver
2. Establish the Database Connection
3. Create a Statement Object
4. Execute a Query
5. Process the Results
6. Close the Connection

```
// Step 1: Load the Driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Step 2: Establish Connection
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/myDB",
"root", "password");

// Step 3: Create Statement
Statement stmt = con.createStatement();

// Step 4: Execute Query
ResultSet rs = stmt.executeQuery("SELECT * FROM students");

// Step 5: Process Result
while(rs.next()) System.out.println(rs.getString("name"));

// Step 6: Close Resource
con.close();
```

JDBC CONNECTION URL FORMAT

- URL Structure:

```
jdbc:mysql://localhost:3306/myDatabase
```

jdbc:	mysql	localhost:3306	myDatabase
JDBC prefix	DB type	Host & Port	DB Name

URL Examples for Different Databases:

Database	JDBC URL	Port
MySQL	jdbc:mysql://localhost:3306/dbname	3306
Oracle	jdbc:oracle:thin:@localhost:1521:xe	1521
PostgreSQL	jdbc:postgresql://localhost:5432/dbname	5432
SQL Server	jdbc:sqlserver://localhost:1433;databaseName=db	1433
SQLite	jdbc:sqlite:database.db	N/A

- You must download and add the appropriate JDBC Driver JAR file to your project classpath before connecting. Example: mysql-connector-java-8.x.jar

CREATING DATABASE & TABLE WITH JDBC – PART 1

```
import java.sql.*;
public class CreateDBTable {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        System.out.println("Driver Loaded ! Connected to MySQL !");

        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/", "root", "password");
        Statement stmt = con.createStatement();

        stmt.executeUpdate("CREATE DATABASE IF NOT EXISTS StudentDB");
        System.out.println("Database Created !");
        stmt.executeUpdate("USE StudentDB");
        stmt.executeUpdate("CREATE TABLE IF NOT EXISTS students (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100),
         age INT, marks DOUBLE, city VARCHAR(100))");
        System.out.println("Table Created !");
```

CREATING DATABASE & TABLE WITH JDBC – PART 2

```
        PreparedStatement ps = con.prepareStatement("INSERT INTO students (name, age, marks, city) VALUES (?,?,?,?)");
        ps.setString(1, "Alice"); ps.setInt(2, 20); ps.setDouble(3, 92.5); ps.setString(4, "Mumbai");
        ps.executeUpdate();
        ps.setString(1, "Bob"); ps.setInt(2, 22); ps.setDouble(3, 87.0); ps.setString(4, "Delhi"); ps.executeUpdate();
        ps.setString(1, "Charlie"); ps.setInt(2, 21); ps.setNull(3, Types.DOUBLE); ps.setString(4, null);
        ps.executeUpdate();
        System.out.println("Records Inserted !\n");

        ResultSet rs = stmt.executeQuery("SELECT * FROM students");
        System.out.println("ID\tNAME\tAGE\tMARKS\tCITY");
        while(rs.next()) {
            System.out.println(rs.getInt("id") + "\t" + rs.getString("name") + "\t" + rs.getInt("age") + "\t" +
            rs.getDouble("marks") + "\t" + rs.getString("city"));
        }

        ps.close(); con.close();
    }
}
```

JDBC PROGRAM – OUTPUT & EXPLANATION

Output:

```
Driver Loaded!        Connected to MySQL!
Database Created!     Table Created!      Records Inserted!
ID     NAME         AGE    MARKS     CITY
1      Alice        20     92.5      Mumbai
2      Bob          22     87.0      Delhi
3      Charlie      21     95.5      Pune
Connection Closed!
```

Code	Explanation
`Class.forName(DRIVER)`	Dynamically loads JDBC Driver class into JVM
`DriverManager.getConnection`	Creates physical connection to MySQL database
`con.createStatement()`	Creates Statement object to send SQL to DB
`CREATE DATABASE IF NOT EXISTS`	Creates DB only if it doesn't already exist
`AUTO_INCREMENT`	MySQL auto-generates unique ID for each record
`executeUpdate()`	Used for DDL (CREATE) and DML (INSERT/UPDATE/DELETE)
`executeQuery()`	Used for SELECT – returns ResultSet
`rs.next()`	Moves cursor to next row; false when no more rows

PREPAREDSTATEMENT – PARAMETERIZED QUERIES

- Definition:
  - **Prepared Statement** is a pre-compiled SQL statement that accepts **parameters at run- time** using ? placeholders. It is **safer** (prevents SQL Injection) and **faster** for repeated execution.

```
import java.sql.*;
public class PreparedStatementDemo {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/StudentDB", "root", "password");
        PreparedStatement ps = con.prepareStatement("INSERT INTO students (name, age, marks, city) VALUES (?,?,?,?)");
        ps.setString(1, "Diana"); ps.setInt(2, 23); ps.setDouble(3, 88.5); ps.setString(4, "Chennai");
        ps.executeUpdate(); System.out.println("Record 1 Inserted !");
        ps.setString(1, "Bob"); ps.setInt(2, 22); ps.setDouble(3, 75.0); ps.setString(4, "Delhi"); ps.executeUpdate();
        System.out.println("Record 2 Inserted !");
        ps.setString(1, "Evan"); ps.setInt(2, 21); ps.setDouble(3, 82.5); ps.setString(4, "Bangalore");
        ps.executeUpdate(); System.out.println("Record 3 Inserted !");
        ResultSet rs = con.createStatement().executeQuery("SELECT name, age, city FROM students ORDER BY name");
        while (rs.next()) System.out.println(rs.getString("name") + " | Age: " + rs.getInt("age") + " | City: " +
        rs.getString("city"));
        ps.close(); con.close();
    }
}
```

PREPAREDSTATEMENT – OUTPUT & COMPARISON

- Output:

```
Record 1 Inserted!
Record 2 Inserted!
Bob     | Age: 22 | City: Delhi
Diana   | Age: 23 | City: Chennai
Evan    | Age: 21 | City: Bangalore
```

Statement vs Prepared Statement:

Feature	Statement	PreparedStatement
SQL Compilation	Compiled every time	Pre-compiled once ✓
Parameters	Hard-coded	Uses ? placeholders ✓
SQL Injection	in SQL	Safe ✓
Performance	Vulnerable Slower (repeated SQL)	Faster for repeated use ✓
Reusability	Low	High ✓
Best For	One-time, simple queries	Dynamic, repeated queries

SUMMARY – JDBC

Topic	Key Points
JDBC	Java API for database connectivity (`java.sql.*`)
Driver Types	Type 1: JDBC-ODBC, Type 2: Native, Type 3: Network, Type 4: Pure Java
Best Driver	Type 4 (Pure Java / Thin Driver) – most popular
Architecture	App → JDBC API → Driver Manager → Driver → Database
Key Classes 6	`DriverManager`, `Connection`, `Statement`, `PreparedStatement`, `ResultSet`
Basic Steps	Load Driver → Connect → Create Stmt → Execute → Process → Close
executeQuery	For SELECT – returns `ResultSet`
executeUpdate	For INSERT/UPDATE/DELETE/DDL – returns `int`
PreparedStmt	Parameterized, pre-compiled, safe from SQL Injection
Best Practice: Always close JDBC resources in `finally` block (or use try-with-resources) to prevent connection leaks.	

QUICK REFERENCE – CHEAT SHEET

- JDBC Essentials

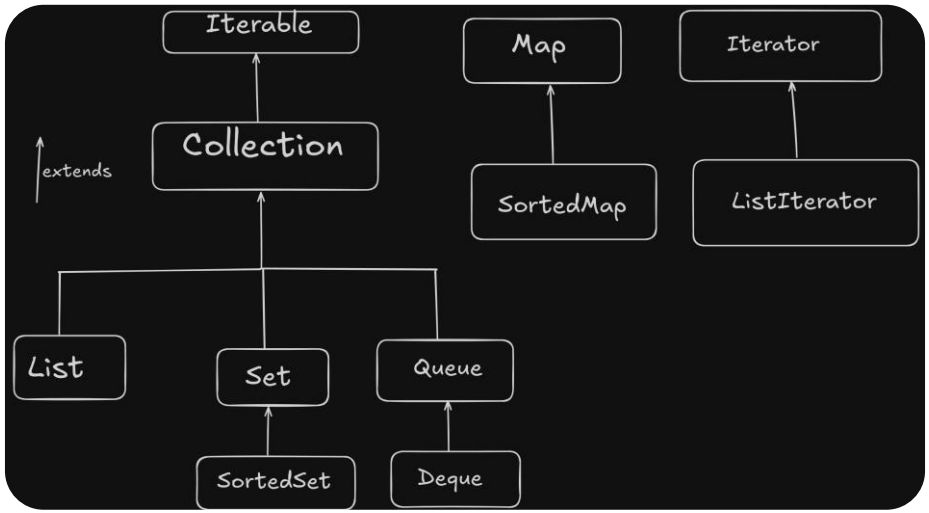
```
1 Class . forName (" driver ");
2 Connection con =
3 DriverManager . getConnection (
4 url , user , pass );
5 Statement s =
6 con . createStatement ();
7 s. executeQuery (" SELECT ...");
8 s. executeUpdate (" INSERT ... ");
9 PreparedStatement ps =
10 con . prepareStatement ( sql );
11 ps. setString (1, " value ");
12 ps. setInt (2, 100) ;
13 rs. next ();
14 rs. getString (" col");
15 rs. close (); s. close ();
16 con . close ();
```

COLLECTIONS - OVERVIEW

- A Collection is a framework providing ready-made data structures to store, manipulate, and retrieve groups of objects dynamically.
- **Array vs. Collection**

Array	Collection
Fixed size Same type only No methods.	Dynamic size Stores Objects Rich methods.

Hierarchy:



COLLECTION INTERFACES

Interface	Key Characteristics
Collection	Root interface: add, remove, size
List	Ordered, allows duplicates, indexed access. Impl: ArrayList, LinkedList, Vector
Set	No duplicates, unordered. Impl: HashSet, TreeSet
Queue	FIFO. Impl: LinkedList, PriorityQueue
Deque	Double-ended queue. Impl: ArrayDeque
Map	Key-Value pairs, no duplicate keys. Impl: HashMap, TreeMap

Important Methods

```
add(E e)              remove(Object o)
contains(Object o)    size()
isEmpty()             clear()
iterator()            toArray()
```

LIST INTERFACE

- **List Interface**
  - Ordered collection (maintains insertion order)
  - Allows **duplicate elements**
  - Elements accessed by **index** (like an array)
  - Part of java.util package
- **Common Implementations:**

Class	Feature
`ArrayList`	Resizable array, fast random access
`LinkedList`	Doubly-linked list, fast insert/delete
`Vector`	Synchronized (thread-safe) ArrayList
`Stack`	LIFO (Last-In-First-Out) structure

ARRAY LIST

- **Definition:**
  - ArrayList is a resizable array implementation of the List interface that allows dynamic growth and shrinkage, maintaining insertion order and allowing duplicate elements.
- **Features**
  - Dynamic array (resizes automatically)
  - Maintains insertion order
  - Allows duplicate elements
  - Allows multiple null values
  - Not synchronized (not thread-safe)
  - Fast random access (index-based)

Important Methods	
add(E e)	Appends element to the end
add(int index, E e)	Inserts element at specified index
get(int index)	Returns element at specified index
set(int index, E e)	Replaces element at index
remove(int index)	Removes element at index
size()	Returns number of elements
contains(Object o)	Checks if element exists
clear()	Removes all elements

ARRAY LIST - EXAMPLE

```
import java.util.*;
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");  list.add("Banana");
        list.add("Cherry"); list.set(1, "Mango");
        System.out.println(list);
    }
}

OUTPUT:
-------
  [Apple, Mango, Cherry]
```

LINKED LIST

- **Definition**
  - LinkedList is a doubly-linked list implementation of List and Deque interfaces, allowing efficient insertion and deletion from both ends.
- **Features**
  - Implemented as doubly-linked list
  - Implements both List and Deque
  - Fast insertion/deletion (O(1)) at ends
  - Slower random access (O(n))
  - Allows duplicates and null elements
  - Not synchronized

IMPORTANT METHODS	
ADDFIRST(E E)	Inserts element at the beginning
ADDLAST(E E)	Inserts element at the end
REMOVEFIRST()	Removes first element
REMOVELAST()	Removes last element
GETFIRST()	Returns first element
GETLAST()	Returns last element
PEEK()	Retrieves head without removing
POLL()	Retrieves and removes head

LINKED LIST – EXAMPLE

```java
import java.util.*;
public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Dog"); list.add("Cat");
        list.addFirst("Bird"); list.addLast("Fish");
        System.out.println(list);
    }
}

OUTPUT:
-------
  [Bird, Dog, Cat, Fish]
```

VECTOR

- **Definition**
  - Vector is a legacy class that implements a growable array of objects, functioning like ArrayList but with synchronized (thread-safe) methods.

Important Methods	
add(E e)	Adds element to the vector
get(int index)	Returns element at index
remove(int index)	Removes element at index
elementAt(int index)	Returns element (legacy method)
firstElement()	Returns first element
lastElement()	Returns last element
size()	Returns number of elements

- **Features**
  - Legacy class (since Java 1.0)
  - Synchronized (thread-safe)
  - Maintains insertion order
  - Allows duplicates and null values
  - Slower than ArrayList due to synchronization

VECTOR - EXAMPLE

```java
import java.util.*;
public class VectorDemo {
    public static void main(String[] args) {
        Vector<Integer> vector = new Vector<>();
        vector.add(10); vector.add(20);
        vector.add(30); vector.remove(1);
        System.out.println(vector);
    }
}

OUTPUT:
-------
  [10, 30]
```

STACK

- **Definition**
  - Stack is a class that extends Vector and implements a Last-In-First-Out (LIFO) data structure.

- **Features**
  - Extends Vector class
  - Follows LIFO principle
  - Synchronized (thread-safe)
  - Allows duplicates and null values

Important Methods	
push(E item)	Pushes item onto the stack
pop()	Removes and returns top element
peek()	Returns top element without removing
empty()	Checks if stack is empty
search(Object o)	Returns position of element from top

STACK - EXAMPLE

```java
import java.util.*;
public class StackDemo {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(10); stack.push(20);
        stack.push(30);
        System.out.println("Pop: " + stack.pop());
        System.out.println(stack);
    }
}

OUTPUT:
-------
  Pop:  30
    [10, 20]
```

HASHSET

- **Definition**
  - HashSet is a class implementing the Set interface, backed by a hash table, that does not allow duplicate elements.
- **Features**
  - No duplicate elements allowed
  - No guaranteed order of elements
  - Allows one null element
  - Backed internally by HashMap
  - Not synchronized

Important Method	
add(E e)	Adds element if not already present
remove(Object o)	Removes specified element
contains(Object o)	Checks if element exists
size()	Returns number of elements
isEmpty()	Checks if set is empty
clear()	Removes all elements

HASHSET - EXAMPLE

```java
import java.util.*;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Red");
        set.add("Green");
        set.add("Red");  // duplicate ignored
        System.out.println(set);
    }
}

OUTPUT:
-------
  [Red, Green]
```

LINKEDHASHSET

- **Definition**
  - LinkedHashSet extends HashSet and maintains a linked list of entries, preserving the insertion order of elements.
- **Features**
  - Extends HashSet
  - Maintains insertion order
  - No duplicate elements
  - Allows one null element
  - Slightly more memory overhead than HashSet

Important Methods	
add(E e)	Adds element maintaining order
remove(Object o)	Removes specified element
contains(Object o)	Checks if element exists
iterator()	Returns iterator in insertion order
size()	Returns number of elements

LINKEDHASHSET - EXAMPLE

```java
import java.util.*;

public class LinkedHashSetDemo {
    public static void main(String[] args) {
        LinkedHashSet<String> set = new LinkedHashSet<>();
        set.add("Zebra");
        set.add("Apple");
        set.add("Mango");
        System.out.println(set);
    }
}

OUTPUT:
-------
  [Zebra, Apple, Mango]
```

TREESET

- **Definition**
  - TreeSet is a class implementing NavigableSet, storing elements in a sorted (ascending) order using a Red-Black tree.

- **Features**
  - Stores elements in sorted order
  - No duplicate elements
  - Does NOT allow null elements
  - Backed by a Red-Black tree
  - Implements NavigableSet and SortedSet

Important Methods	
add(E e)	Adds element maintaining sort order
first()	Returns smallest element
last()	Returns largest element
higher(E e)	Returns next higher element
lower(E e)	Returns next lower element
headSet(E e)	Returns elements less than e
tailSet(E e)	Returns elements ≥ e

TREESET - EXAMPLE

```java
import java.util.*;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> set = new TreeSet<>();
        set.add(50);
        set.add(10);
        set.add(30);
        System.out.println(set);
        System.out.println("First: " + set.first());
    }
}

OUTPUT:
-------
  [10, 30, 50]
  First: 10
```

PRIORITYQUEUE

- **Definition**
  - PriorityQueue is a queue implementation where elements are ordered based on their natural ordering or a provided Comparator.
- **Features**
  - Elements ordered by priority (min-heap by default)
  - Unbounded queue, backed by an array (binary heap)
  - Does NOT allow null elements
  - Head = smallest element by default
  - Not thread-safe

Important Methods	
add(E e) / offer(E e)	Inserts element into queue
poll()	Retrieves and removes head
peek()	Retrieves head without removing
remove()	Removes head, throws exception if empty
size()	Returns number of elements

PRIORITYQUEUE - EXAMPLE

```
import java.util.*;
public class PriorityQueueDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(50); pq.add(10); pq.add(30);
        System.out.println("Peek: " + pq.peek());
        System.out.println("Poll: " + pq.poll());
    }
}
OUTPUT:
-------
  Peek: 10
  Poll: 10
```

ARRAYDEQUE

- **Definition**
  - ArrayDeque is a resizable-array implementation of the Deque interface, allowing insertion and removal from both ends, usable as both a Stack and a Queue.
- **Features**
  - Implements Deque (Double Ended Queue)
  - Can be used as Stack (LIFO) or Queue (FIFO)
  - Faster than LinkedList/Stack for these operations
  - Does NOT allow null elements
  - Not thread-safe

ARRAYDEQUE – IMPORTANT METHODS

Important Methods	
addFirst(E e)	Inserts element at front
addLast(E e)	Inserts element at end
offer(E e)	Inserts element (as queue)
poll()	Removes head (as queue)
push(E e)	Pushes element (as stack)
pop()	Pops element (as stack)

ARRAYDEQUE - EXAMPLE

```
import java.util.*;

public class ArrayDequeDemo {
    public static void main(String[] args) {
        ArrayDeque<Integer> deque = new ArrayDeque<>();
        deque.offer(10);
        deque.offer(20);
        deque.push(5);
        System.out.println(deque);
    }
}
OUTPUT:
-------
  [5, 10, 20]
```

HASHMAP

- **Definition**
  - HashMap is a class implementing the Map interface that stores data as key-value pairs using a hash table for storage.
- **Features**
  - Stores key-value pairs
  - Keys are unique; values can duplicate
  - No guaranteed order
  - Allows one null key, multiple null values
  - Not synchronized

HASHMAP – IMPORTANT METHODS

Important Methods	
put(K key, V value)	Inserts key-value pair
get(Object key)	Returns value for given key
remove(Object key)	Removes entry for key
containsKey(Object key)	Checks if key exists
keySet()	Returns set of all keys
values()	Returns collection of all values
entrySet()	Returns set of key-value mappings

HASHMAP - EXAMPLE

```java
import java.util.*;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("Alice", 25);
        map.put("Bob", 30);
        System.out.println(map);
        System.out.println("Bob: " + map.get("Bob"));
    }
}

OUTPUT:
-------
  {Bob=30, Alice=25}
  Bob: 30
```

LINKEDHASHMAP

- **Definition**
  - LinkedHashMap extends HashMap and maintains a doubly-linked list of entries, preserving the insertion order of keys.
- **Features**
  - Allows one null key, multiple null values
  - Slightly slower than HashMap due to ordering
  - Useful for LRU Cache implementations
  - Extends HashMap
  - Maintains insertion order

Important Methods	
put(K key, V value)	Inserts key-value pair
get(Object key)	Returns value for given key
remove(Object key)	Removes entry for key
keySet()	Returns keys in insertion order
entrySet()	Returns entries in insertion order

LINKEDHASHMAP - EXAMPLE

```java
import java.util.*;

public class LinkedHashMapDemo {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
        map.put("Zebra", 1);
        map.put("Apple", 2);
        map.put("Mango", 3);
        System.out.println(map);
    }
}

OUTPUT:
-------
  {Zebra=1, Apple=2, Mango=3}
```

TREEMAP

- **Definition**
  - TreeMap is a class implementing NavigableMap that stores key-value pairs in sorted order based on keys, using a Red-Black tree.
- **Features**
  - Keys stored in sorted (ascending) order
  - Implements NavigableMap and SortedMap
  - Does NOT allow null keys
  - Backed by a Red-Black tree
  - Not synchronized

Important Methods	
put(K key, V value)	Inserts key-value pair
firstKey()	Returns smallest key
lastKey()	Returns largest key
higherKey(K key)	Returns next higher key
lowerKey(K key)	Returns next lower key
headMap(K key)	Returns entries with keys < key

TREEMAP - EXAMPLE

```java
import java.util.*;

public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap<String, Integer> map = new TreeMap<>();
        map.put("Zebra", 1);
        map.put("Apple", 2);
        map.put("Mango", 3);
        System.out.println(map);
        System.out.println("First Key: " + map.firstKey());
    }
}

OUTPUT:
-------
  {Apple=2, Mango=3, Zebra=1}
  First Key: Apple
```

STREAM API - NEED FOR STREAM API

- **Definition**
  - The Stream API (introduced in Java 8) is used to process collections of objects in a functional, declarative style, allowing operations like filtering, mapping, and reducing without explicit iteration.
- **Why Do We Need It? (Problems with Traditional Approach)**
  - Traditional loops require verbose, imperative code
  - Difficult to write parallel processing code manually
  - No built-in support for chaining operations
  - Hard to express "what to do" vs "how to do it"
  - Stream API solves this using functional programming concepts (lambda expressions)

STREAM API – KEY BENEFITS

Feature	Benefit
Declarative	Focus on "what" not "how"
Chainable	Multiple operations combined fluently
Lazy Evaluation	Operations executed only when needed
Parallel Processing	Easy with parallelStream()
Immutability	Doesn't modify original collection

STREAM API - EXAMPLE

```
WITHOUT STREAM API
------------------
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evenSquares = new ArrayList<>();
for (Integer n : nums) {
    if (n % 2 == 0) {
        evenSquares.add(n * n);
    }
}
System.out.println(evenSquares);
-------------------------------------------------------
WITH STREAM API
---------------
import java.util.*;
import java.util.stream.*;
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evenSquares = nums.stream()
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .collect(Collectors.toList());
System.out.println(evenSquares);

OUTPUT:
-------
  [4, 16, 36]
```

STREAM API - OPERATIONS

Method	Type	Description
filter(Predicate)	Intermediate	Selects elements matching a given condition
map(Function)	Intermediate	Transforms each element (1-to-1 mapping)
flatMap(Function)	Intermediate	Flattens nested streams into a single stream
distinct()	Intermediate	Removes duplicate elements
sorted()	Intermediate	Sorts elements in natural or custom order
peek(Consumer)	Intermediate	Performs an action on each element (for debugging)
limit(long n)	Intermediate	Restricts stream to first n elements
skip(long n)	Intermediate	Skips the first n elements
forEach(Consumer)	Terminal	Performs action on each element (order not guaranteed)
forEachOrdered(Consumer)	Terminal	Performs action on each element (order guaranteed)
reduce(identity, accumulator)	Terminal	Combines elements into a single result
toArray()	Terminal	Converts stream into an array
collect(Collector)	Terminal	Converts stream into a List, Set, Map, or String

INTERMEDIATE VS TERMINAL OPERATIONS

Category	Characteristic	Examples
Intermediate	Returns a new Stream; Lazy (not executed until terminal op is called); Can be chained	filter, map, flatMap, distinct, sorted, peek, limit, skip
Terminal	Produces a final result (not a Stream); Triggers execution of the pipeline; Stream cannot be reused after	forEach, forEachOrdered, reduce, toArray, collect

STREAM API OPERATIONS - EXAMPLE

```java
import java.util.*;
import java.util.stream.*;

public class StreamAllOperationsDemo {
    public static void main(String[] args) {

        List<List<Integer>> nestedNumbers = Arrays.asList(
                Arrays.asList(5, 10, 15),
                Arrays.asList(20, 10, 25),
                Arrays.asList(30, 35, 5)
        );

        System.out.println("---- Stream Pipeline Execution ----");
```

CONTD...

```
List<Integer> result = nestedNumbers.stream()
        .flatMap(list -> list.stream())      // flatten nested lists
        .distinct()                          // remove duplicates
        .filter(n -> n > 5)                  // keep numbers > 5
        .map(n -> n * 2)                     // double each number
        .peek(n -> System.out.println("After map: " + n))  // debug print
        .sorted()                            // sort ascending
        .skip(1)                             // skip first element
        .limit(4)                            // take only 4 elements
        .collect(Collectors.toList());       // collect into List

System.out.println("\nFinal Result List: " + result);

// forEach - print each element
System.out.println("\n---- forEach ----");
result.forEach(n -> System.out.println("Value: " + n));

// forEachOrdered - guarantees order (useful in parallel streams)
System.out.println("\n---- forEachOrdered (parallel) ----");
result.parallelStream()
      .forEachOrdered(n -> System.out.println("Ordered: " + n));
```

CONTD...

```
        // reduce - sum of all elements
        int sum = result.stream()
                .reduce(0, (a, b) -> a + b);
        System.out.println("\nSum using reduce(): " + sum);

        // toArray - convert stream to array
        Integer[] array = result.stream().toArray(Integer[]::new);
        System.out.println("Array: " + Arrays.toString(array));

        // collect - join as comma-separated string
        String joined = result.stream()
                .map(String::valueOf)
                .collect(Collectors.joining(", "));
        System.out.println("Joined String: " + joined);
    }
}
```

STREAM API OPERATIONS - OUTPUT

```
---- Stream Pipeline Execution ----
After map: 20
After map: 30
After map: 40
After map: 50
After map: 60
After map: 70

Final Result List: [30, 40, 50, 60]


Sum using reduce(): 180
Array: [30, 40, 50, 60]
Joined String: 30, 40, 50, 60
```

```
---- forEach ----
Value: 30
Value: 40
Value: 50
Value: 60

---- forEachOrdered (parallel) ----
Ordered: 30
Ordered: 40
Ordered: 50
Ordered: 60
```

STEP-BY-STEP BREAKDOWN (CODE)

Step	Operation	Effect on Data
1	flatMap()	Combines all inner lists → [5,10,15,20,10,25,30,35,5]
2	distinct()	Removes duplicates → [5,10,15,20,25,30,35]
3	filter(n > 5)	Keeps values > 5 → [10,15,20,25,30,35]
4	map(n * 2)	Doubles each value → [20,30,40,50,60,70]
5	peek()	Prints each value (debug only, doesn't change data)
6	sorted()	Sorts ascending → [20,30,40,50,60,70]
7	skip(1)	Removes first element → [30,40,50,60,70]
8	limit(4)	Keeps first 4 → [30,40,50,60]
9	collect()	Converts to List<Integer>